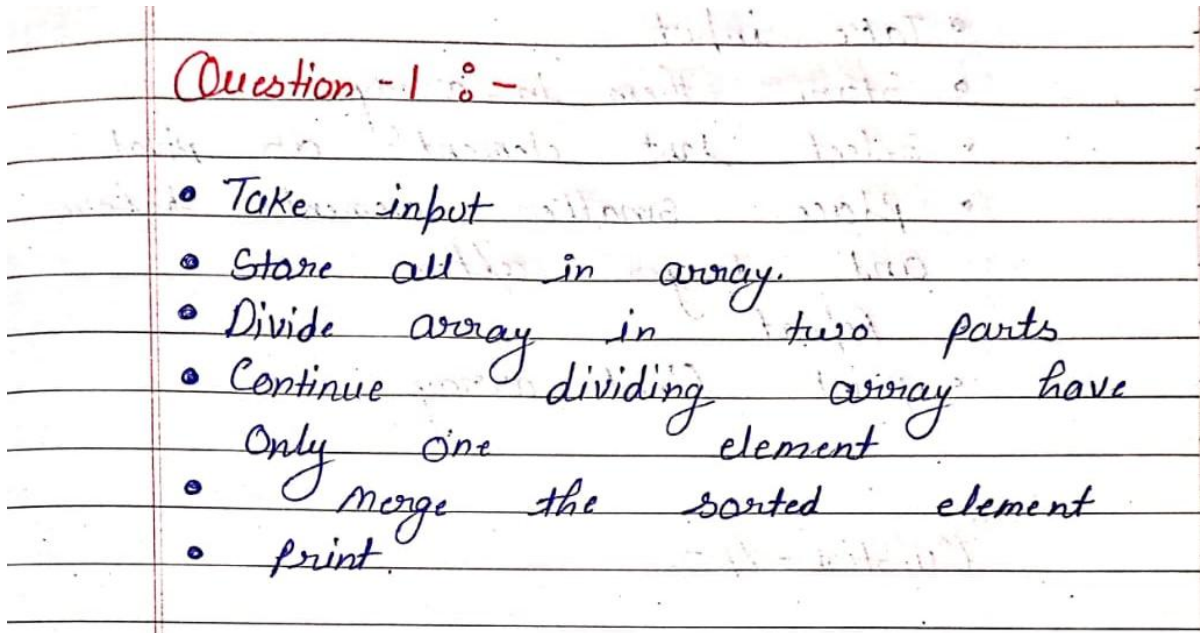


Week 3 Module

Question 01



```
import java.util.Scanner;

public class merge_sort {

    static void merge(int arr[], int left, int mid, int right) {

        int n1 = mid - left + 1;
        int n2 = right - mid;

        int L[] = new int[n1];
        int R[] = new int[n2];

        for (int i = 0; i < n1; i++)
            L[i] = arr[left + i];

        for (int j = 0; j < n2; j++)
            R[j] = arr[mid + 1 + j];

        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {
```

```

        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}

static void mergeSort(int arr[], int left, int right) {

    if (left < right) {

        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();

    int arr[] = new int[n];

    for (int i = 0; i < n; i++)
        arr[i] = sc.nextInt();

    mergeSort(arr, 0, n - 1);

    for (int x : arr)
        System.out.print(x + " ");
}
}

```

```

    at merge_sort.main(merge_sort.java:52)
PS C:\Users\Neeraj\Desktop\college\code> cd "c:\Users\Neeraj\Desktop\col
t }
6
450 1200 800 650 300 1500
300 450 650 800 1200 1500 PS C:\Users\Neeraj\Desktop\college\code> 

```

Question 02

Question - 2 :-

- Take input.
- Store in array.
- Sort the array.
- Print array.
- Find median.
- Count processing time.
- Calculate difference b/t max min.
- Print.

```
import java.util.Scanner;

public class medium {

    static void merge(int arr[], int left, int mid, int right) {

        int n1 = mid - left + 1;
        int n2 = right - mid;

        int L[] = new int[n1];
        int R[] = new int[n2];

        for (int i = 0; i < n1; i++)
            L[i] = arr[left + i];

        for (int j = 0; j < n2; j++)
            R[j] = arr[mid + 1 + j];

        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {
            if (L[i] <= R[j])
                arr[k++] = L[i++];
            else
                arr[k++] = R[j++];
        }
    }
}
```

```

        while (i < n1)
            arr[k++] = L[i++];

        while (j < n2)
            arr[k++] = R[j++];
    }

    static void mergeSort(int arr[], int left, int right) {

        if (left < right) {

            int mid = (left + right) / 2;

            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);

            merge(arr, left, mid, right);
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int arr[] = new int[n];

        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();

        mergeSort(arr, 0, n - 1);

        // Sorted Array
        for (int x : arr)
            System.out.print(x + " ");

        System.out.println();

        // Median
        int median;
        if (n % 2 == 1)
            median = arr[n / 2];
        else

```

```

        median = (arr[n / 2] + arr[(n / 2) - 1]) / 2;

        System.out.println("Median: " + median);

        // Orders Above Median
        int count = 0;

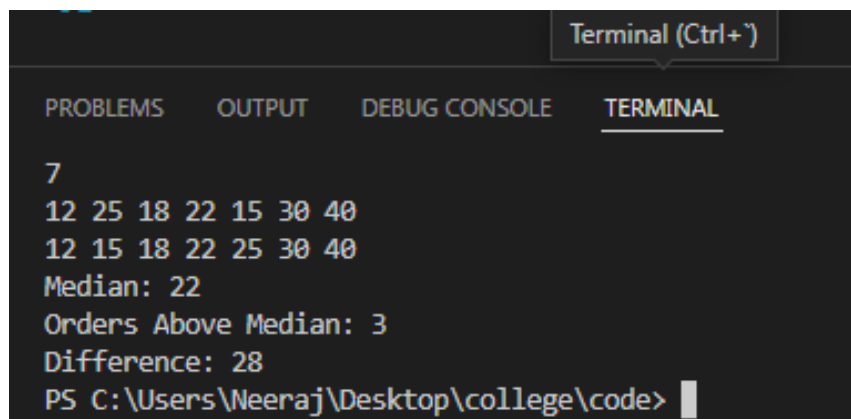
        for (int x : arr)
            if (x > median)
                count++;

        System.out.println("Orders Above Median: " + count);

        // Difference
        int difference = arr[n - 1] - arr[0];

        System.out.println("Difference: " + difference);
    }
}

```



```

Terminal (Ctrl+`)
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

7
12 25 18 22 15 30 40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
PS C:\Users\Neeraj\Desktop\college\code>

```

Question 03

Question 3-

- Take input
- Store them in array
- Select last element as pivot
- Place smaller element before and larger after
- Repeat
- Print final array.

```
import java.util.Scanner;

public class quick_sort_easy {

    static int partition(int arr[], int low, int high) {

        int pivot = arr[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {

            if (arr[j] < pivot) {
                i++;

                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;
    }
}
```

```

        return i + 1;
    }

    static void quickSort(int arr[], int low, int high) {

        if (low < high) {

            int pi = partition(arr, low, high);

            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int arr[] = new int[n];

        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();

        quickSort(arr, 0, n - 1);

        for (int x : arr)
            System.out.print(x + " ");

    }
}

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

        at java.base/java.util.Scanner.nextInt(Scanner.j
        at quick_sort_easy.main(quick_sort_easy.java:48)
PS C:\Users\Neeraj\Desktop\college\code> cd "c:\Users\Ne
k_sort_easy }
8
120 14 17 45 78 200 60 95
14 17 45 60 78 95 120 200 PS C:\Users\Neeraj\Desktop\col

```

Question 04

Question - 4 -

- Take input
- Store in array.
- Store them in descending order.
- Print array.
- Display top five value
- Calculate average top five
- Calculate average over all
- Count and print.

```
import java.util.Scanner;

public class quick_short_mid {

    static int partition(int arr[], int low, int high) {

        int pivot = arr[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {

            if (arr[j] > pivot) {        // Descending order
                i++;

                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
```



```
        arr[high] = temp;

        return i + 1;
    }

    static void quickSort(int arr[], int low, int high) {

        if (low < high) {

            int pi = partition(arr, low, high);

            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int arr[] = new int[n];

        long sum = 0;

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
            sum += arr[i];
        }

        quickSort(arr, 0, n - 1);

        // Sorted Array
        for (int x : arr)
            System.out.print(x + " ");

        System.out.println();

        // Top 5
        System.out.print("Top 5: ");

        long topSum = 0;

        for (int i = 0; i < 5; i++) {
```

```

        System.out.print(arr[i] + " ");
        topSum += arr[i];
    }

    System.out.println();

    System.out.println("Average of Top 5: " + (topSum / 5));

    double overallAverage = (double) sum / n;

    int count = 0;

    for (int x : arr)
        if (x > overallAverage)
            count++;

    System.out.println("Values Above Overall Average: " + count);
}
}

```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL
			<pre> 8 500 1200 700 2000 900 1500 3000 2500 3000 2500 2000 1500 1200 900 700 500 Top 5: 3000 2500 2000 1500 1200 Average of Top 5: 2040 Values Above Overall Average: 3 PS C:\Users\Neeraj\Desktop\college\code> </pre>

Question 05

Question - 5 -

- Take input
- Store in array.
- Build a max Heap from array.
- Swap root with last ele.
- Reduce the heap size
- Repeat
- Print final array.

```
import java.util.Scanner;

public class heap_sort_easy {

    static void heapify(int arr[], int n, int i) {

        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left] > arr[largest])
            largest = left;

        if (right < n && arr[right] > arr[largest])
            largest = right;

        if (largest != i) {

            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;

            heapify(arr, n, largest);
        }
    }
}
```

```

static void heapSort(int arr[]) {

    int n = arr.length;

    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Heap Sort
    for (int i = n - 1; i > 0; i--) {

        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();

    int arr[] = new int[n];

    for (int i = 0; i < n; i++)
        arr[i] = sc.nextInt();

    heapSort(arr);

    for (int x : arr)
        System.out.print(x + " ");

}
}

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

Average of Top 5: 2040
Values Above Overall Average: 3
PS C:\Users\Neeraj\Desktop\college\code> cd "c:\Users\Neeraj\Desktop\college\code"
sort_easy }
6
850 1200 950 600 1500 1000
600 850 950 1000 1200 1500 PS C:\Users\Neeraj\Desktop\college\code>

```

Question 06

Question -6 -

- Take input
- Store in array.
- Sort the array.
- Print sorted array.
- find fastest / slowest response time.
- Calculate Average response time.
- Count the case faster the average
- Calculate and display the percentage.

```
import java.util.Scanner;

public class heap_sort_mmid {

    static void heapify(int arr[], int n, int i) {

        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left] > arr[largest])
            largest = left;

        if (right < n && arr[right] > arr[largest])
            largest = right;

        if (largest != i) {

            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;
        }
    }
}
```

```

        heapify(arr, n, largest);
    }
}

static void heapSort(int arr[]) {

    int n = arr.length;

    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {

        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();

    int arr[] = new int[n];

    double sum = 0;

    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
        sum += arr[i];
    }

    heapSort(arr);

    // Sorted Array
    for (int x : arr)
        System.out.print(x + " ");

    System.out.println();
}

```

```

        System.out.println("Fastest: " + arr[0]);
        System.out.println("Slowest: " + arr[n - 1]);

        double average = sum / n;
        System.out.printf("Average: %.2f\n", average);

        int count = 0;

        for (int x : arr) {
            if (x < average)
                count++;
        }

        System.out.println("Cases Faster Than Average: " + count);

        double percentage = (count * 100.0) / n;
        System.out.printf("Percentage: %.2f%%", percentage);
    }
}

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
va } ; if ($?) { java heap_sort_mid }
8
12 7 15 9 20 5 10 8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%PS C:\Users\Neeraj\Desktop\college\code>

```